

19 | Vault（三）：一站式证书管理解决方案

潘野 · 云原生基础架构实战课



你好，我是潘野。

前面我们已经学习了如何使用 Vault 管理密钥，以及如何使用 Vault 来安全地传递密码。

今天的课程里，我们将会学习如何使用 Vault 来管理企业内部的证书，特别是 TLS 证书。我们在工作中应该都接触过 TLS 证书，它也叫做 SSL 证书，这个证书用于数据的加密传输，可以保证数据在传输过程中的隐私性和完整性，帮助用户完成身份验证。

通过今天的学习，你不但能更好地理解管理 TLS 证书的要点，还可以掌握如何利用 Vault 工具来管理这些证书，确保企业数据传输的安全性。

TLS 证书

尽管许多人都在使用 TLS 证书，但他们可能对 TLS 证书的工作原理、申请和使用流程等细节并不十分了解。

在实际的工作环境中，无论是开发新的应用、维护现有系统还是进行网络配置，几乎百分百都要和 TLS 证书打交道。比方说，系统管理员需要定期检查和更新证书，开发人员需要在应用程序中实现正确的 TLS 配置，而安全专家则需要审核和评估 TLS 的使用情况以确保安全合规。

除了系统管理员和网络管理员需要深入了解 TLS 协议的细节之外，对于大多数人而言，他们更多接触的是 TLS 证书的管理过程。而证书管理的第一环就是申请证书。

申请 TLS 证书的流程

想要申请一个 TLS 证书并不复杂，整个过程非常简单，一共就 2 步。

第一步，创建证书签名请求，英文叫 Certificate Signing Request（后面我们简称为 CSR）。是在申请 TLS/SSL 证书时必需的一个文件。它包含了申请证书所需的关键信息，包括你的公钥以及一些标识信息，如组织名称、地理位置、部门名称等。这些信息用于在证书中确认你或者你所在组织的身份。

创建 CSR 文件有几种方式，一种是通过 openssl 命令行的方式。另外一种是直接使用在线的服务，比如在 <https://csrgenerator.com/> 上直接填入相关的信息即可。你可以参考后面这张图，展示的是如何为 cloudnative.example.com 这个域名申请一张证书。

Generate a Certificate Signing Request

Complete this form to generate a new CSR and private key.

Country

CN

State

Shanghai

Locality

PuDong

Organization

JKSJ

Organizational Unit

Cloudnative

Common Name

cloudnative.example.com

Alternative Names

foo.example.com, bar.example.com

Key Size

☒ 2048 ☐ 4096

Generate CSR

点击“Generate CSR”后，你将看到生成了一个 CSR 文件和一个 Private Key 文件。就像后面的示例代码这样，Private Key 文件是你申请证书的私钥。我想提醒你注意的是，**这个私钥极其重要，绝不能外泄！**

 复制代码

```
1  -----BEGIN CERTIFICATE REQUEST-----
2  MIIC8jCCAdoCAQAwDELMAkGA1UEBhMCQ04xETAPBgNVBAGMCFNoYW5naGFpMQ8w
3  ....
4  4nRs9bvdNiPhcY5WBKB2ZXngPqLN9EKAj3h8eVgMtWUncZTQT0k=
5  -----END CERTIFICATE REQUEST-----
6
7  -----BEGIN PRIVATE KEY-----
8  MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBAcwggSjAgEAAoIBAQCdcmF5cfdEwNTy
9  ....
10 mybnQ5RWqKF01nS0uaBp23c7hNJxP/kHQtfJqBqT4mb7YHpQtVYGlv2QBg7i8o1y
11 TjDz2fDXxoQ0spW0sYMXCW0=
12 -----END PRIVATE KEY-----
```

接下来进行第二步，携带第一步生成的 CSR 文件到证书颁发机构，即我们常说的 CA 机构，去申请一张证书。

那么这里的证书颁发机构都会完成哪些事情呢？

首先要做身份验证。在颁发证书之前，CA 需要验证申请证书的实体（可以是个人、公司或设备）的身份。通俗来说，你提交的信息里需要证明这个 domain 是属于你的！

接下来就是证书颁发。一旦验证了申请者的身份，CA 就会创建一个数字证书，证书中包含了申请者的公钥和其他身份信息，并且用 CA 的私钥对其进行签名。这个签名保证了证书的真实性和完整性。

除此之外，CA 还要负责证书撤销和信任链维护。

先来看**证书撤销**。如果证书被发现有误，或者密钥被泄露，或者证书持有者不再需要该证书，CA 就要负责撤销证书。撤销的证书会被加入到 CA 发布的证书撤销列表（CRL）中。

然后是**信任链到维护**。CA 还负责维护一个信任链，这意味着它们的签名需要被其他信任的 CA 认证。顶级 CA（根 CA）的证书通常内置在操作系统、浏览器等中，作为信任的根源。

证书颁发机构

我们常见的证书颁发机构包括 Let's Encrypt、DigiCert、GoDaddy 等，它们提供的证书可在公网上使用。这类证书的优势在于操作系统已预装了这些机构的 CA 根证书，因此在各种电脑上均可避免证书不被信任的问题。

另一类是企业内部的 CA，即企业自建和管理的 CA 系统，用于发行和管理数字证书。对多数企业而言，建立内部 CA 具有以下几点优势。

首先，在成本控制方面，自建和运营 CA 的成本通常低于购买公共 CA 的证书，特别是当企业需为大量用户、设备或应用程序颁发证书时。从长远来看，内部 CA 能够显著降低成本。

其次是灵活性。在云原生架构中，微服务已成主流，服务间的通信需通过 TLS 加密确保安全，而建立了内部 CA 以后，这些通信就不必依赖公网证书了。

第三是安全性。由于 CA 是内部管理，一旦私钥泄露，就可以迅速吊销受影响的证书。

证书的管理难点

证书的生命周期管理涉及从创建、部署到过期和更新的每一个环节，都需要精确执行。我们常常需要管理数百甚至数千个证书，每个证书都具有不同的有效期和用途。

在大型企业中，证书可能被用于多个应用和服务，这些应用和服务可能位于不同的物理位置和网络环境。不同的技术栈和平台（例如云服务和本地服务器）使得管理更加复杂。

管理大规模证书环境的一大挑战就是缺乏自动化机制。自动化对于证书的自动续签、部署和撤销至关重要，但许多企业仍依赖低效的手动过程管理，这不仅非常耗时，还可能因操作错误增加安全风险。

我们这就来聊一聊 Vault 是如何处理证书自动化管理问题的。

配置 Vault 管理证书

Vault 的 PKI (Public Key Infrastructure) 密钥引擎提供了完整的 X.509 证书签发和管理功能。它可以作为一个内部 CA (证书颁发机构) 来签发证书, 或者作为一个中间 CA 来代理其他 CA 的证书签发。

1. 设置 Vault 作为 CA

首先, 我们需要在 Vault 中启用 PKI 秘密引擎, 并配置它为根 CA 或中间 CA。这涉及到创建和配置 CA 的证书和私钥。

 复制代码

```
1 vault secrets enable pki
2 vault secrets tune -max-lease-ttl=87600h pki
3 vault write pki/root/generate/internal common_name="example.com" ttl=87600h
```

2. 配置证书签发角色

角色是 Vault 中定义的一种策略, 用于控制证书的签发。通过配置角色, 可以详细定义证书的参数, 如有效期、允许的域名等。

 复制代码

```
1 vault write pki/roles/example-dot-com allowed_domains="example.com" allow_subdoma
```

3. 签发和管理证书

一旦角色配置完成, 就可以根据这些角色签发证书了。应用程序可以通过 API 调用直接向 Vault 请求证书, 之后 Vault 将按照预定义的角色策略签发证书。

 复制代码

```
1 vault write pki/issue/example-dot-com common_name="www.example.com"
```

自动化

虽然 Vault 提供了强大的证书管理和安全存储功能，但要在现代云原生环境中实现证书的完全自动化管理，我们还需要进一步整合和自动化工作流。这就是为什么将 Vault 和 Cert-Manager 这样的工具集成非常关键，特别是在 Kubernetes 环境中。

Cert-Manager 使用插件式架构，允许它通过 “Issuer” 资源与各种外部系统集成。这些 Issuer 配置定义了如何与特定的 CA 进行通信，包括 Vault。每种类型的 Issuer 都有自己的配置需求和参数，适用于相应的后端系统。

Cert-Manager 与 Vault 的交互是通过 Vault 的 HTTP API 完成的。Cert-Manager 根据 `Certificate` 资源中定义的参数（如公钥、主题、有效期等）构造 API 请求，Vault 将会处理这些请求并返回签发的证书。接着，Cert-Manager 将这些证书存储在 Kubernetes Secrets 中，以供使用。

接下来，我们看看怎么将 Cert-Manager 与 vault 集成在一起，集成的大致步骤如下。

首先在 Vault 中启用 PKI 引擎，配置 CA 证书和角色。这样就 Vault 就成为了一个中心化的证书颁发机构，负责生成和签发证书。

接下来在 Vault 中创建一个 Kubernetes 身份验证角色，允许 Cert-Manager 使用该角色访问 Vault。

最后，在 Kubernetes 中安装 Cert-Manager，并创建一个 Issuer（证书颁发者）资源，指向 Vault 的 PKI 引擎。注意 Issuer 需要使用上一步创建的角色进行身份验证。

创建资源的代码如下所示。

 复制代码

```
1 apiVersion: cert-manager.io/v1
2 kind: Issuer
3 metadata:
4   name: vault-issuer
5   namespace: cert-manager
6 spec:
```

```
7  vault:
8    path: pki/sign/cert-manager-role
9    server: https://<VAULT_SERVER_ADDRESS>
10   caBundle: <BASE64_ENCODED_CA_CERT>
11   auth:
12     kubernetes:
13       role: cert-manager
14       mountPath: /v1/auth/kubernetes
15       secretRef:
16         name: vault-token
17         key: token
18
```

这段代码里有两处需要你根据自己的情况做替换。

替换 `<VAULT_SERVER_ADDRESS>` 为你的 Vault 服务地址。

替换 `<BASE64_ENCODED_CA_CERT>` 为你的 Vault 服务器 CA 证书的 Base64 编码值。

如何向 Vault 申请证书

首先在 Kubernetes 中创建 Certificate 资源，指定使用上一步的 Issuer。Cert-Manager 会自动向 Vault 申请证书，并且将证书和私钥存储在 Kubernetes 的 Secret 中。

 复制代码

```
1  apiVersion: cert-manager.io/v1
2  kind: Certificate
3  metadata:
4    name: example-com
5    namespace: default
6  spec:
7    secretName: example-com-tls
8    duration: 2160h # 90d
9    renewBefore: 360h # 15d
10   subject:
11     organizations:
12       - Example Inc.
13   commonName: example.com
14   isCA: false
15   privateKey:
```

```
16     algorithm: RSA
17     encoding: PKCS1
18     size: 2048
19     usages:
20       - server auth
21       - client auth
22     dnsNames:
23       - example.com
24       - www.example.com
25     issuerRef:
26       name: vault-issuer
27       kind: Issuer
28       group: cert-manager.io
29
```

这个 Certificate 资源定义了我们想要的证书的各种属性，我们重点关注后面这几个字段。

`commonName`：证书的公用名。

`duration`：证书的有效期，这个非常非常重要！

`renewBefore`：证书到期前多久进行续期。

`usages`：证书的用途。

`dnsNames`：证书的 DNS 名称。

`issuerRef`：指定使用哪个 Issuer 来签发证书。

创建这个 Certificate 资源后。Cert-Manager 会自动向 Vault 申请证书。Cert-Manager 使用访问令牌，调用 Vault PKI 引擎的签发接口，提交证书签发请求。

Cert-Manager 收到签发的证书，将其连同私钥一起存储在指定的 Secret 中。这个 Secret 包含两个数据项：`tls.crt`：证书文件和 `tls.key`：私钥文件，应用可以直接挂载这个 Secret，使用其中的证书和私钥来启用 HTTPS。

总结

在今天的课程中，我们深入探讨了如何使用 Vault 来管理企业内部的证书，特别是 TLS 证书。TLS 证书是用于确保数据在传输过程中的安全性，包括加密、数据完整性和身份验证。通

过 Vault 的 PKI 功能，我们可以有效管理证书从创建、发行、撤销和更新的整个生命周期。

使用 Vault 作为 CA 的主要优势之一在于其卓越的安全性和审计功能。因为 Cert-Manager 能够与 Vault 进行安全通信，所以用 Cert-Manager 从 Vault 获取证书，就会按照 Vault 的 API 规范来申请、续签和管理证书。这样的集成为 Kubernetes 环境中的证书管理提供了一种高效、灵活的自动化方案。

在实际操作中，使用 Vault 管理 TLS 证书的过程包括几个关键步骤。首先，需要在 Vault 中设置并配置 PKI 后端，这一步骤包括生成根证书或者导入现有的根证书。接下来，定义一套角色策略，这些策略将控制证书的各种属性，如有效期、使用域名等。

总之，Vault 提供了一个全面的、安全的解决方案，用于管理 TLS 证书的整个生命周期。通过其 PKI 功能和与其他工具如 Cert-Manager 的集成，Vault 能够满足最严格的安全需求，是企业进行证书管理的理想选择。

思考题

除了 Cert Manager，你还有哪些手段可以从 Vault 中自动获取证书？

欢迎在评论区与我讨论。如果这一讲对你有启发，也欢迎分享给身边更多朋友。

AI智能总结

1. TLS证书的申请流程包括创建证书签名请求（CSR）和携带CSR到证书颁发机构申请证书。
2. 证书颁发机构需要进行身份验证，并签发包含申请者公钥和其他身份信息的数字证书。
3. 企业内部的CA系统具有成本控制、灵活性和安全性等优势，适用于大规模证书管理。
4. 证书的生命周期管理涉及创建、部署、过期和更新，需要精确执行，但缺乏自动化机制是一大挑战。
5. Vault的PKI密钥引擎提供完整的X.509证书签发和管理功能，可作为内部CA或中间CA来签发证书。
6. 配置Vault作为CA需要启用PKI秘密引擎、配置证书和私钥，并设置证书签发角色。
7. Cert-Manager与Vault的集成是关键，特别是在Kubernetes环境中，通过Issuer资源定义与特定CA的通信方式。
8. Cert-Manager使用插件式架构，允许通过Issuer资源与各种外部系统集成，包括Vault。
9. 在Vault中启用PKI引擎、配置CA证书和角色，创建Kubernetes身份验证角色，允许Cert-Manager使用该角色访问Vault。
10. Cert-Manager通过访问令牌调用Vault PKI引擎的签发接口，提交证书签发请求，将签发的证书存储在Kubernetes Secrets中。

精选留言

由作者筛选后的优质留言将会公开显示，欢迎踊跃留言。